Practical File

of

Fundamentals of C Programming

(24CSE0107)

Batch-2025

Bachelor of Engineering (CSE)

Submitted By
Student Name - Akshyat Kumar
Roll No. 2510991394
Group No., Sem: 2nd, BE (CSE)
Chitkara University, Punjab, 140401
India

Submitted To
Faculty Name - Tejpal Sharma
Designation, CSE
Chitkara University, Punjab, 140401
India

Department of
Computer Science and Engineering,
Chitkara University School of Engineering and Technology,
Chitkara University, Punjab, India

Index

Experiment No. 1.

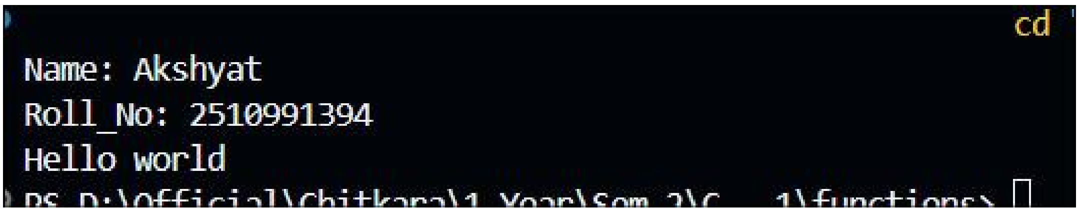
Aim: Install C compiler (GCC/Code::Blocks), set up IDE, compile and run the first "Hello, World!" program.

Concept Used: The concept used is the basic structure of a C program, compiler setup, and the use of stdio.h for printing output.

Program:

```
#include<stdio.h>

int main(){
    printf("Name: Akshyat\n");
    printf("Roll_No: 2510991394\n");
    printf("Hello world\n");
   return 0;
}
```

Output Screenshot:



Experiment No. 2.

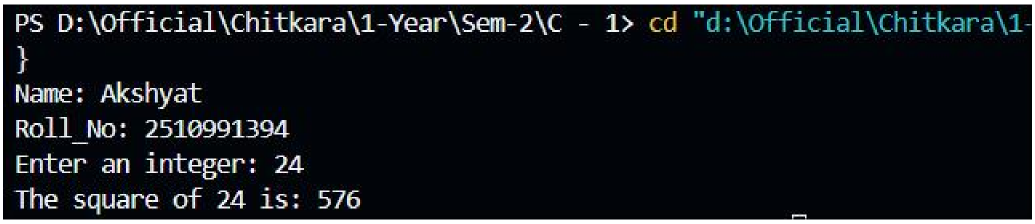
Aim: Write a Program to show the use to input (Scanf)/output (Printf) statements and block structure of C-program by highlighting the features of "stdio.h".

Concept Used: The concept used is input/output handling in C using scanf and printf, along with block structure demonstration.

Program:

```
#include <stdio.h>

int main() {


    printf("Name: Akshyat\n");
    printf("Roll No: 2510991394\n");

    int number, square;

    // Output statement
    printf("Enter an integer: ");

    //Input statement
    scanf("%d", &number);

    square = number * number;

    printf("The square of %d is: %d\n", number, square);

    return 0;
}
```

Output Screenshot:



Experiment No. 3.

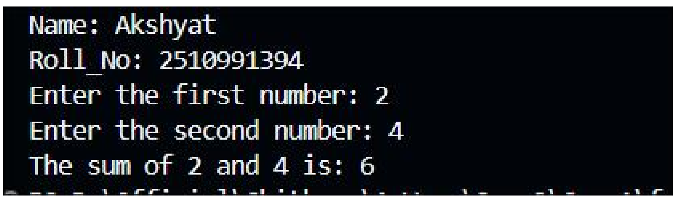
Aim: Write a program to add two numbers and display the sum.

Concept Used: The concept used is arithmetic addition of two integers with user input and output display.

Program:

```
#include <stdio.h>

int main() {


    printf("Name: Akshyat\n");
    printf("Roll No: 2510991394\n");

    int num1, num2, sum;

    // Taking frist number from the user
    printf("Enter the first number: ");
    scanf("%d", &num1);

    // Taking second number from the user
    printf("Enter the second number: ");
    scanf("%d", &num2);


    sum = num1 + num2;

    // Displaying the result
    printf("The sum of %d and %d is: %d\n", num1, num2, sum);

    return 0;
}
```

Output Screenshot:



Experiment No. 4.

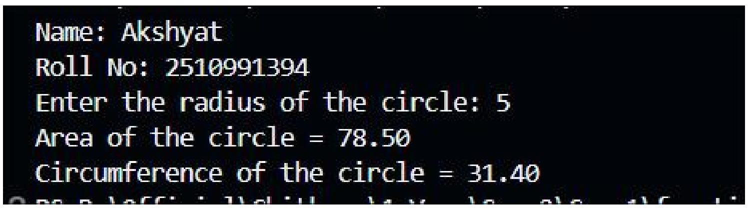
Aim: Write a program to calculate the area and the circumference of a circle by using radius as the input provided by the user.

Concept Used: The concept used is applying mathematical formulas to calculate area and circumference of a circle using floating-point arithmetic.

Program:

```
#include <stdio.h>

int main() {
    printf("Name: Akshyat\n");
    printf("Roll No: 2510991394\n");

    float radius, area, circumference;
    // Taking radius as input
    printf("Enter the radius of the circle: ");
    scanf("%f", &radius);

    // Calculating area and circumference
    area = 3.14 * radius * radius;
    circumference = 2 * 3.14 * radius;

    // Displaying results
    printf("Area of the circle = %.2f\n", area);
    printf("Circumference of the circle = %.2f\n", circumference);
    return 0;
}
```

Output Screenshot:



Experiment No. 5.

Aim: Write a Program to perform addition, subtraction, division and multiplication of two numbers given as input by the user.

Concept Used: The concept used is performing arithmetic operations (addition, subtraction, multiplication, division) on user-provided numbers.

Program:

```
#include <stdio.h>

int main() {

    printf("Name: Akshyat\n");
    printf("Roll No: 2510991394\n");

    float num1, num2;
    float add, sub, mul, div;

    // Take the first number from the user
    printf("Enter the first number: ");
    scanf("%f", &num1);

    // Take the second number from the user
    printf("Enter the second number: ");
    scanf("%f", &num2);

    // Perform the arithmetic operations
    add = num1 + num2;
    sub = num1 - num2;
    mul = num1 * num2;
    div = num1 / num2;

    // Display the final results (%.2f limits the output to 2 decimal places)
    printf("\n--- Results ---\n");
    printf("Addition: %.2f\n", add);
    printf("Subtraction: %.2f\n", sub);
    printf("Multiplication: %.2f\n", mul);
    printf("Division: %.2f\n", div);

    return 0;
}
```

Output Screenshot:

```
$?) { .\5 }
Name: Akshyat
Roll No: 2510991394
Enter the first number: 8
Enter the second number: 4

--- Results ---
Addition: 12.00
Subtraction: 4.00
Multiplication: 32.00
Division: 2.00
```

Experiment No. 6.

Aim: Write a program to evaluate each of the following equations. (i) $V = u + at$. (ii) $S = ut+1/2at^2$ (iii) $T=2*a+\sqrt{b}+9c$ (iv) $H=\sqrt{b^2+p^2}$

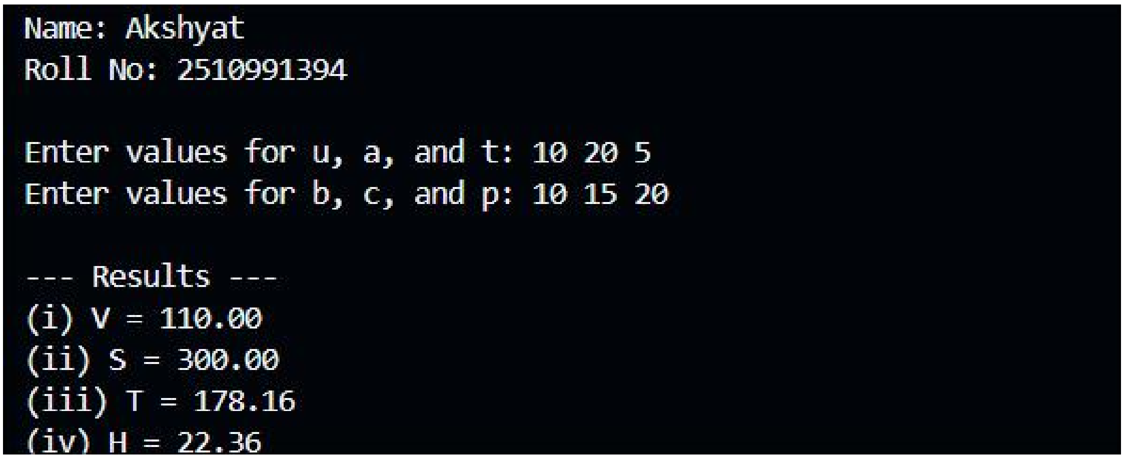
Concept Used: The concept used is evaluating mathematical equations using operators and <math.h> functions like sqrt.

Program:

```
#include <stdio.h>

#include <math.h> // Included for the sqrt() function

int main() {

printf("Name: Akshyat\n");

printf("Roll No: 2510991394\n\n");


float u, a, t, b, c, p;

float V, S, T, H;


// Take inputs for the variables required in the equations

printf("Enter values for u, a, and t: ");

scanf("%f %f %f", &u, &a, &t);


printf("Enter values for b, c, and p: ");

scanf("%f %f %f", &b, &c, &p);


// Evaluate the equations

V = u + (a * t);

S = (u * t) + (0.5 * a * t * t);

T = (2 * a) + sqrt(b) + (9 * c);
```

```c
H = sqrt((b * b) + (p * p));


// Display the final results (%.2f limits the output to 2 decimal places)

printf("\n--- Results ---\n");

printf("(i) V = %.2f\n", V);

printf("(ii) S = %.2f\n", S);

printf("(iii) T = %.2f\n", T);

printf("(iv) H = %.2f\n", H);

return 0;


}
```

Output Screenshot:



```
Name: Akshyat
Roll No: 2510991394

Enter values for u, a, and t: 10 20 5
Enter values for b, c, and p: 10 15 20

--- Results ---
(i) V = 110.00
(ii) S = 300.00
(iii) T = 178.16
(iv) H = 22.36
```

Experiment No. 7.

Aim: Write a program to swap two variables:

a) By using temporary variable.
b) Without using temporary variable

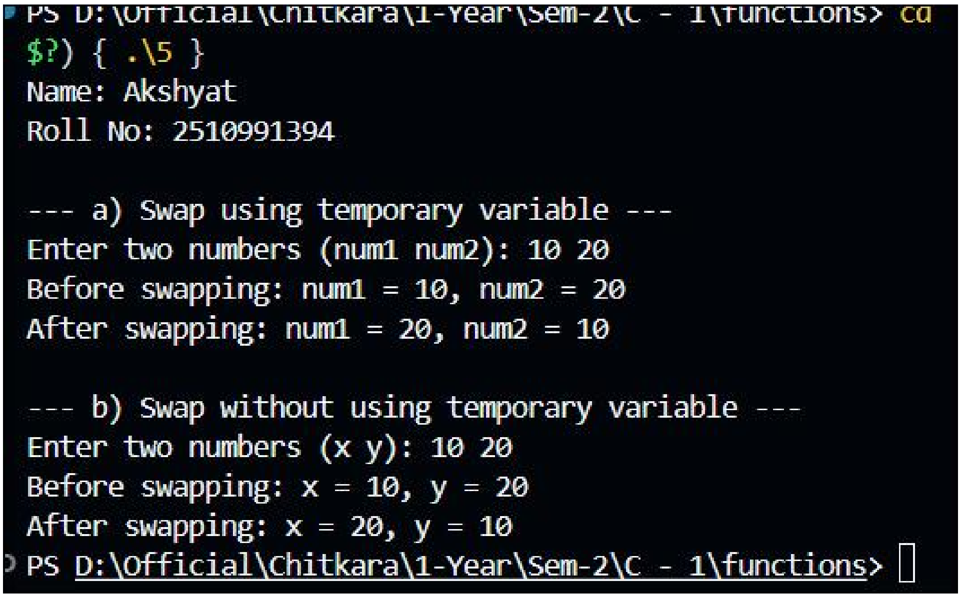
Concept Used: The concept used is swapping variables, both with a temporary variable and without using one (arithmetic swapping).

Program:

```
#include <stdio.h>


int main() {


    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");


    int num1, num2, temp;

    int x, y;


    // a) By using temporary variable

    printf("--- a) Swap using temporary variable ---\n");

    printf("Enter two numbers (num1 num2): ");

    scanf("%d %d", &num1, &num2);


    printf("Before swapping: num1 = %d, num2 = %d\n", num1, num2);


    temp = num1;

    num1 = num2;
```

```c
 num2 = temp;

    printf("After swapping: num1 = %d, num2 = %d\n\n", num1, num2);

    // b) Without using temporary variable

    printf("--- b) Swap without using temporary variable ---\n");

    printf("Enter two numbers (x y): ");

    scanf("%d %d", &x, &y);
    printf("Before swapping: x = %d, y = %d\n", x, y)
    x = x + y;
    y = x - y;

x = x - y;

    printf("After swapping: x = %d, y = %d\n", x, y);

    return 0;
}
```

Output Screenshot:



```
PS D:\Official\Chitkara\1-Year\Sem-2\C - 1\functions> cd
$?) { .\5 }
Name: Akshyat
Roll No: 2510991394

--- a) Swap using temporary variable ---
Enter two numbers (num1 num2): 10 20
Before swapping: num1 = 10, num2 = 20
After swapping: num1 = 20, num2 = 10

--- b) Swap without using temporary variable ---
Enter two numbers (x y): 10 20
Before swapping: x = 10, y = 20
After swapping: x = 20, y = 10
PS D:\Official\Chitkara\1-Year\Sem-2\C - 1\functions>
```

Experiment No. 8.

Aim: Write a Program to find the greatest among three numbers using:
Conditional Operator
If-Else statement

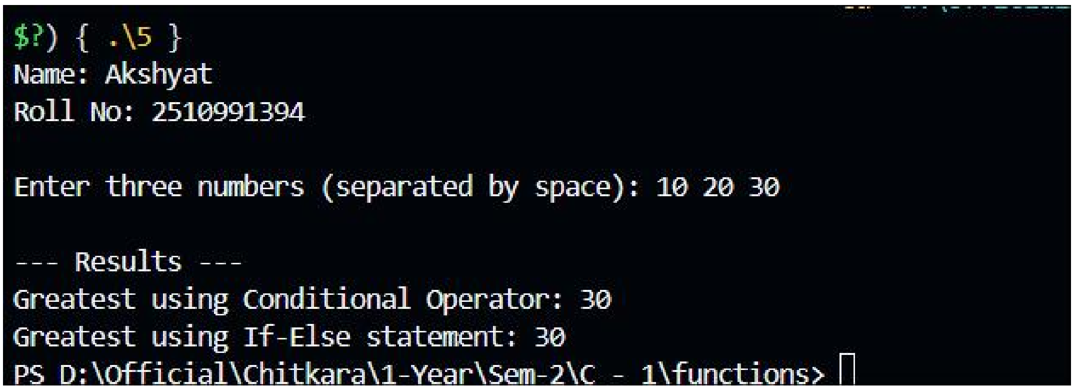
Concept Used: The concept used is finding the greatest among three numbers using conditional (ternary) operators and if-else statements.

Program:

```
#include <stdio.h>


int main() {


    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");


    int num1, num2, num3;

    int max_conditional, max_ifelse;


    // Take three numbers from the user

    printf("Enter three numbers (separated by space): ");

    scanf("%d %d %d", &num1, &num2, &num3);


    // 1. Find greatest using Conditional (Ternary) Operator

    max_conditional = (num1 > num2) ? ((num1 > num3) ? num1 : num3) : ((num2 > num3) ?
num2 : num3);


    // 2. Find greatest using If-Else statement

    if (num1 >= num2 && num1 >= num3) {
```

```c
        max_ifelse = num1;
    } else if (num2 >= num1 && num2 >= num3) {
        max_ifelse = num2;
    } else {
        max_ifelse = num3;
    }

    // Display the final results
    printf("\n--- Results ---\n");
    printf("Greatest using Conditional Operator: %d\n", max_conditional);
    printf("Greatest using If-Else statement: %d\n", max_ifelse);

    return 0;
}
```

Output Screenshot:



```
$?) { .\5 }
Name: Akshyat
Roll No: 2510991394

Enter three numbers (separated by space): 10 20 30

--- Results ---
Greatest using Conditional Operator: 30
Greatest using If-Else statement: 30
PS D:\Official\Chitkara\1-Year\Sem-2\C - 1\functions>
```

Experiment No. 9.

Aim: Write the following programs using switch case statement:

To check that an input alphabet is vowel or consonant

To check whether a number is positive, negative or zero

Concept Used: The concept used is decision-making with switch statements to check vowels/consonants and classify numbers as positive, negative, or zero

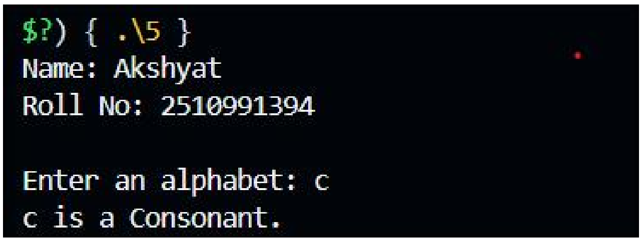
Program:

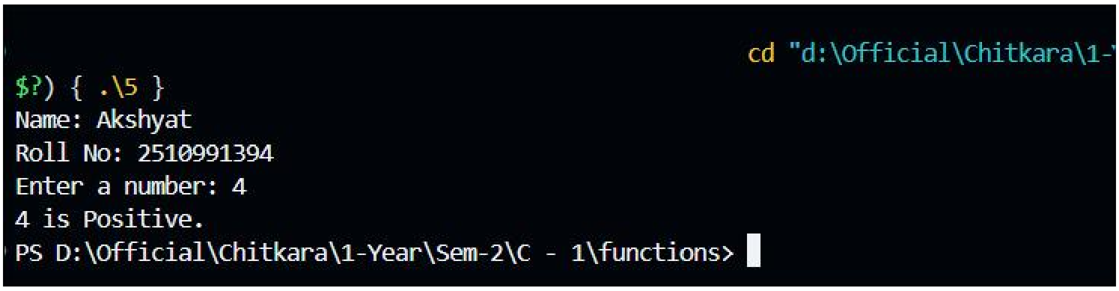
```
// Program to check if input is an consonant or an vowel

#include <stdio.h>

int main() {

    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");

    char ch;

    // Take an alphabet from the user

    printf("Enter an alphabet: ");

    scanf(" %c", &ch);

    // Check for vowels using switch case

    switch (ch) {

        case 'a':

        case 'A':

        case 'e':

        case 'E':

        case 'i':

        case 'I':

        case 'o':
```

```
        case 'O':
        case 'u':
        case 'U':
            printf("%c is a Vowel.\n", ch);
            break;
        default:
            printf("%c is a Consonant.\n", ch);
            break;}
    return 0;
}
// Program to check whether the number is positive,negative or zero
#include <stdio.h>
int main() {
    printf("Name: Akshyat\n");
    printf("Roll No: 2510991394\n");
    int num;
    // Take a number from the user
    printf("Enter a number: ");
    scanf("%d", &num);
    // Switch evaluates if the number is greater than 0 (1 for true, 0 for false)
    switch (num > 0) {
        case 1:
            printf("%d is Positive.\n", num);
            break;
        case 0:
```

```
        // Nested switch evaluates if the number is less than 0
        switch (num < 0) {
            case 1:
                printf("%d is Negative.\n", num);
                break;
            case 0:
                printf("The number is Zero.\n");
                break;
        }
        break;
    }
    return 0;
}
```

Output Screenshot:





Experiment No. 10.

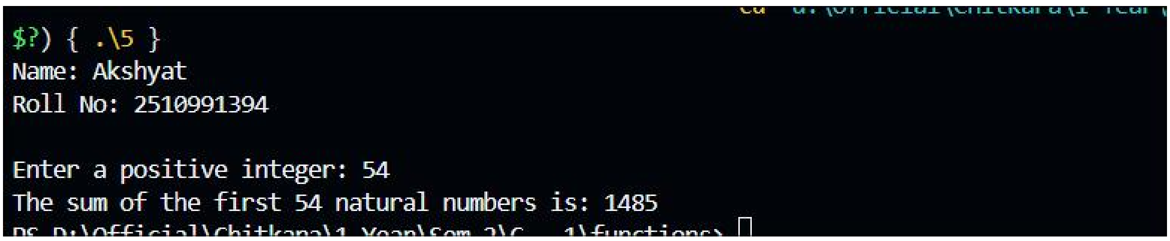
Aim: Write a program using while loop to print the sum of first n natural numbers.

Concept Used: The concept used is iterative computation using a while loop to sum natural numbers.

Program:

```
#include <stdio.h>

int main() {

    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");

     int n, i, sum = 0;

    // Take the value of n from the user

    scanf("%d", &n);

 // Calculate sum using a while loop

    i = 1;

    while (i <= n) {

        sum = sum + i;

        i++; }

    printf("The sum of the first %d natural numbers is: %d\n", n, sum);

    return 0;

}
```

Output Screenshot:



Experiment No. 11.

Aim: Write a program to check a number is Armstrong or not using For loop.

Concept Used: The concept used is checking Armstrong numbers using a for loop and modulus operator.

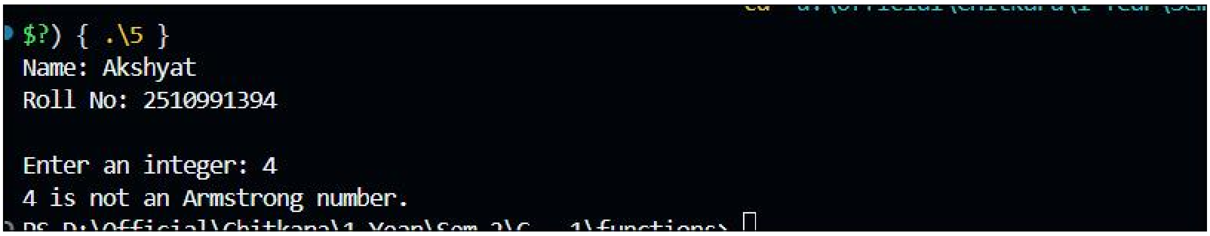
Program:

```
#include <stdio.h>

int main() {

printf("Name: Akshyat\n");

printf("Roll No: 2510991394\n\n");

int num, temp, rem, sum = 0;

// Take a number from the user

printf("Enter an integer: ");

scanf("%d", &num);

// Calculate sum of cubes of digits using a for loop

for (temp = num; temp > 0; temp /= 10) {

    rem = temp % 10;

    sum = sum + (rem * rem * rem);

}

// Check if the sum is equal to the original number

if (sum == num) {

    printf("%d is an Armstrong number.\n", num);

} else {

    printf("%d is not an Armstrong number.\n", num);

}
```

return 0;

}

Output Screenshot:



Experiment No. 12.

Aim: Write the program to count the digits in a number and then print the reverse of the number also.

Concept Used: The concept used is digit manipulation with loops to count digits and reverse a number.

Program:

```
#include <stdio.h>

 int main() {

      printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n");

  int num, count = 0, reverse = 0, remainder;

   // Take a number from the user

    printf("Enter an integer: ");

    scanf("%d", &num);

    // Calculate the number of digits and the reverse

    while (num != 0) {

        remainder = num % 10;

        reverse = reverse * 10 + remainder;

        count++;

        num /= 10;

    }

     // Display the final results

    printf("\n--- Results ---\n");

    printf("Number of digits: %d\n", count);

    printf("Reversed number: %d\n", reverse);
```

```
    return 0;

}
```

Output Screenshot:

```
$?) { .\5 }
Name: Akshyat
Roll No: 2510991394
Enter an integer: 1817

--- Results ---
Number of digits: 4
Reversed number: 7181
```

Experiment No. 13.

Aim: Write a program to generate the Fibonacci series.

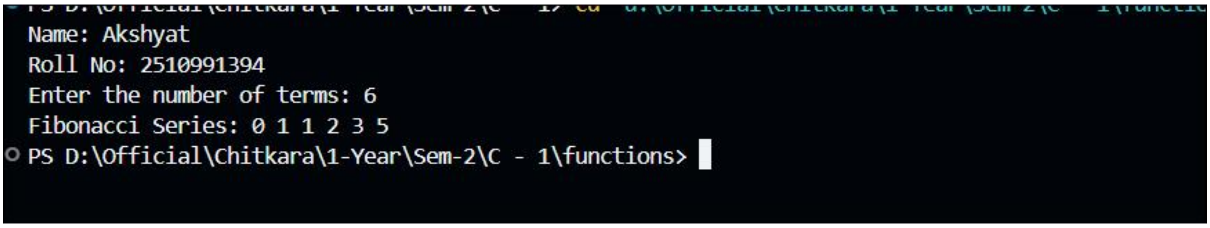
Concept Used: The concept used is a loop to build the series step by step, storing previous values and adding them to get the next term.

Program:

```
#include <stdio.h>

int main() {

    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n");


    int n, i;

    int t1 = 0, t2 = 1, nextTerm;


    // Take the number of terms from the user

    printf("Enter the number of terms: ");

    scanf("%d", &n);


    printf("Fibonacci Series: ");


    // Loop to generate and print the series

    for (i = 1; i <= n; ++i) {

        printf("%d ", t1);

        nextTerm = t1 + t2;

        t1 = t2;

        t2 = nextTerm;
```

```
    }
    printf("\n");


    return 0;
}
```

Output Screenshot:



Experiment No. 14.

Aim: Write a program to print the following patterns:

a)

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
```

b)

```
          *
        * *
      * * *
    * * * *
  * * * * *
* * * * * *
```

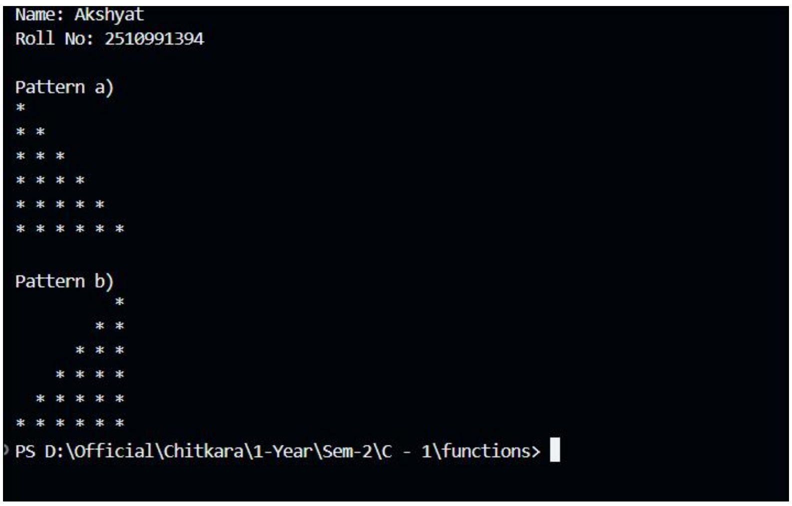
Concept Used: The concept used is nested loops for printing star patterns (left-aligned and right-aligned).

Program:

```
#include <stdio.h>

int main() {

   printf("Name: Akshyat\n");

   printf("Roll No: 2510991394\n\n");

  int i, j, k;

   int rows = 6;

   // Pattern a) - Left-aligned

   printf("Pattern a)\n");

   for (i = 1; i <= rows; i++) {

      for (j = 1; j <= i; j++) {
```

```
            printf("* ");
        printf("\n");
    printf("\n");
 // Pattern b) - Right-aligned
    printf("Pattern b)\n");
    for (i = 1; i <= rows; i++) {
        // Print spaces
        for (k = 1; k <= rows - i; k++) {
            printf("  ");
        }
        // Print stars
        for (j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");}
    return 0;
}
```

Output Screenshot:



Experiment No. 15.

Aim: Write the program to print the following pattern:

1	2	3	4	5	6
2	4	6	8	10	12
3	6	9	12	15	18
4	8	12	16	20	24
5	10	15	20	25	30
6	12	18	24	30	36

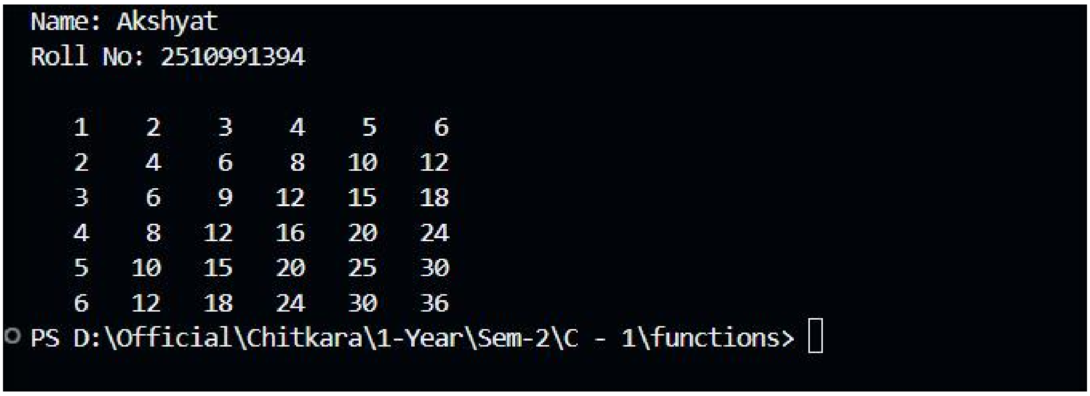
Concept Used: The concept used is nested loops to print multiplication tables in a grid format

Program:

```
#include <stdio.h>


int main() {


    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");


    int i, j;


    // Nested loops to print the multiplication pattern

    for (i = 1; i <= 6; i++) {

        for (j = 1; j <= 6; j++) {

            // %4d ensures the numbers are right-aligned with a width of 4 characters for a clean
grid

            printf("%4d ", i * j);

        }

        printf("\n"); // Move to the next line after each row
```

```
    }

    return 0;
}
```

Output Screenshot:



Experiment No. 16.

Aim: Write a program to perform the following operations on 1D-Array:

- Insert
- Update
- Delete
- Display
- Search

Concept Used: The concept used is array manipulation (insert, update, delete, display, search) using a menu-driven program.

Program:

```
#include <stdio.h>


int main() {


    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");


    int arr[100];

    int size = 0, choice, pos, value, i, found;


    // Initialize the array

    printf("Enter initial number of elements: ");

    scanf("%d", &size);


    printf("Enter the elements: ");

    for (i = 0; i < size; i++) {

        scanf("%d", &arr[i]);
```

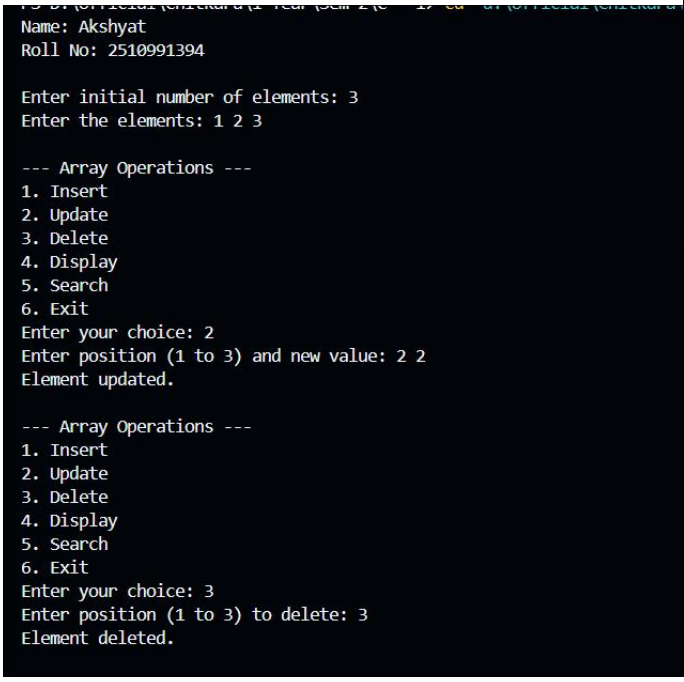
```
}

// Menu-driven loop for array operations
while (1) {
    printf("\n--- Array Operations ---\n");
    printf("1. Insert\n2. Update\n3. Delete\n4. Display\n5. Search\n6. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            // Insert an element
            printf("Enter position (1 to %d) and value: ", size + 1);
            scanf("%d %d", &pos, &value);
            if (pos >= 1 && pos <= size + 1) {
                for (i = size; i >= pos; i--) {
                    arr[i] = arr[i - 1];
                }
                arr[pos - 1] = value;
                size++;
                printf("Element inserted.\n");
            } else {
                printf("Invalid position.\n");
            }
            break;
```

```
case 2:
    // Update an element
    printf("Enter position (1 to %d) and new value: ", size);
    scanf("%d %d", &pos, &value);
    if (pos >= 1 && pos <= size) {
        arr[pos - 1] = value;
        printf("Element updated.\n");
    } else {
        printf("Invalid position.\n");
    }
    break;

case 3:
    // Delete an element
    printf("Enter position (1 to %d) to delete: ", size);
    scanf("%d", &pos);
    if (pos >= 1 && pos <= size) {
        for (i = pos - 1; i < size - 1; i++) {
            arr[i] = arr[i + 1];
        }
        size--;
        printf("Element deleted.\n");
    } else {
        printf("Invalid position.\n");
```

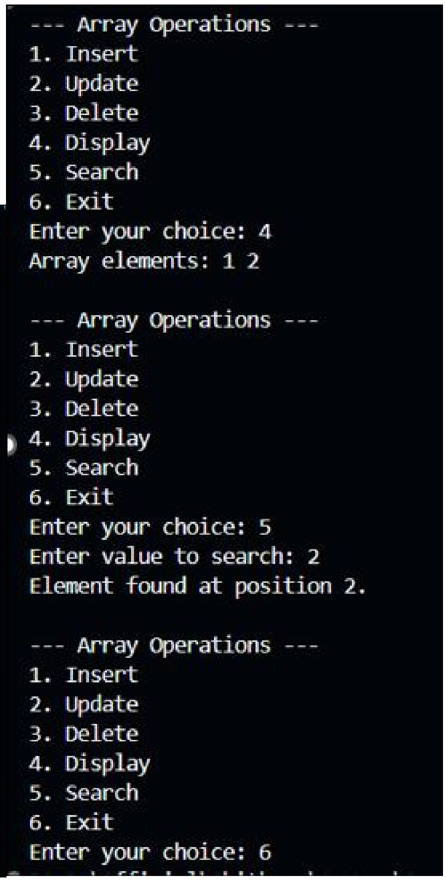
```
    }
    break;

case 4:
    // Display all elements
    printf("Array elements: ");
    for (i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    break;

case 5:
    // Search for an element
    printf("Enter value to search: ");
    scanf("%d", &value);
    found = 0;
    for (i = 0; i < size; i++) {
        if (arr[i] == value) {
            printf("Element found at position %d.\n", i + 1);
            found = 1;
            break;
        }
    }
    if (found == 0) {
```

```c
                printf("Element not found.\n");
        }
            break;
        case 6:
            // Exit the program
            return 0;
        default:
            printf("Invalid choice. Try again.\n");
        }
    }
    return 0;
}
```

Output Screenshot:



```
Name: Akshyat
Roll No: 2510991394

Enter initial number of elements: 3
Enter the elements: 1 2 3

--- Array Operations ---
1. Insert
2. Update
3. Delete
4. Display
5. Search
6. Exit
Enter your choice: 2
Enter position (1 to 3) and new value: 2 2
Element updated.

--- Array Operations ---
1. Insert
2. Update
3. Delete
4. Display
5. Search
6. Exit
Enter your choice: 3
Enter position (1 to 3) to delete: 3
Element deleted.
```



```
--- Array Operations ---
1. Insert
2. Update
3. Delete
4. Display
5. Search
6. Exit
Enter your choice: 4
Array elements: 1 2

--- Array Operations ---
1. Insert
2. Update
3. Delete
4. Display
5. Search
6. Exit
Enter your choice: 5
Enter value to search: 2
Element found at position 2.

--- Array Operations ---
1. Insert
2. Update
3. Delete
4. Display
5. Search
6. Exit
Enter your choice: 6
```

Experiment No. 17.

Aim: Write a program to calculate the sum of array elements by passing it to a function.

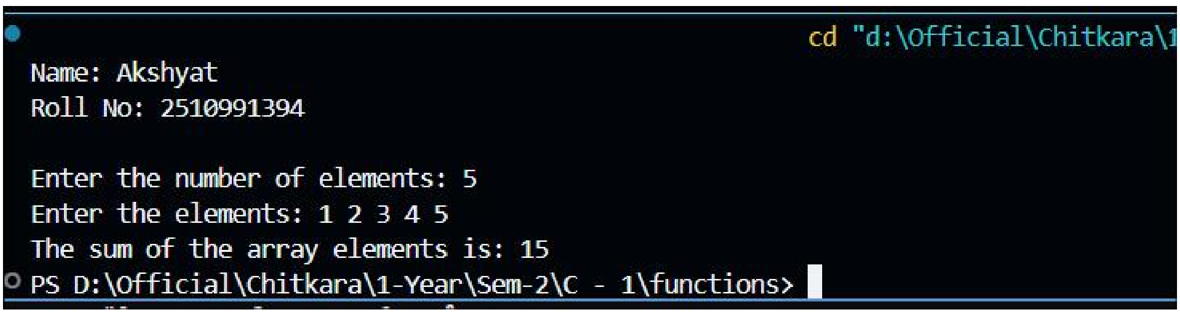
Concept Used: The concept used is passing arrays to functions to calculate the sum of elements.

Program:

```
#include <stdio.h>


// Function to calculate the sum of array elements

int calculateSum(int arr[], int size) {

    int sum = 0;

    for (int i = 0; i < size; i++) {

        sum += arr[i];

    }

    return sum;

}


int main() {


    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");


    int arr[100];

    int size, i, totalSum;


    // Take the size of the array from the user
```

```
    printf("Enter the number of elements: ");
    scanf("%d", &size);

    // Take array elements from the user
    printf("Enter the elements: ");
    for (i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }

    // Call the function and store the result
    totalSum = calculateSum(arr, size);

    // Display the final result
    printf("The sum of the array elements is: %d\n", totalSum);

    return 0;
}
```

Output Screenshot:



Experiment No. 18.

Aim: Write a program matrix multiplication using the concept of 2D array

Concept Used: The concept used is 2D arrays and nested loops for matrix multiplication.

Program:

```
#include <stdio.h>

int main() {

    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");

    int mat1[10][10], mat2[10][10], result[10][10];

    int r1, c1, r2, c2, i, j, k;

    // Take dimensions for the first matrix

    printf("Enter rows and columns for first matrix: ");

    scanf("%d %d", &r1, &c1);

    // Take dimensions for the second matrix

    printf("Enter rows and columns for second matrix: ");

    scanf("%d %d", &r2, &c2);

    // Check if matrix multiplication is possible

    if (c1 != r2) {

        printf("Error: Multiplication not possible. Columns of 1st matrix must equal rows of 2nd
matrix.\n");

        return 0;

    }

    // Input elements for the first matrix

    printf("\nEnter elements of Matrix 1:\n");
```

```
for (i = 0; i < r1; i++) {
    for (j = 0; j < c1; j++) {
        scanf("%d", &mat1[i][j]);
    }}
// Input elements for the second matrix
printf("\nEnter elements of Matrix 2:\n");
for (i = 0; i < r2; i++) {
    for (j = 0; j < c2; j++) {
        scanf("%d", &mat2[i][j]);
    }}
// Initialize the result matrix to 0
for (i = 0; i < r1; i++) {
    for (j = 0; j < c2; j++) {
        result[i][j] = 0;
    }
}

// Perform matrix multiplication
for (i = 0; i < r1; i++) {
    for (j = 0; j < c2; j++) {
        for (k = 0; k < c1; k++) {
            result[i][j] += mat1[i][k] * mat2[k][j];
        }
    }
}
```

```c
    // Display the resultant matrix
    printf("\n--- Resultant Matrix ---\n");
    for (i = 0; i < r1; i++) {
        for (j = 0; j < c2; j++) {
            printf("%d\t", result[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output Screenshot:

```
Name: Akshyat
Roll No: 2510991394

Enter rows and columns for first matrix: 2 2
Enter rows and columns for second matrix: 2 2

Enter elements of Matrix 1:
1 2 3 4

Enter elements of Matrix 2:
5 6 7 8

--- Resultant Matrix ---
19      22
43      50
```

Experiment No. 19.

Aim: Write a program to transpose a given matrix.

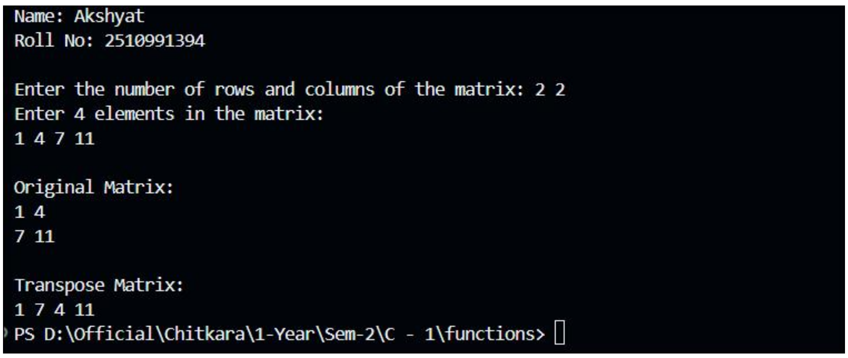
Concept Used: The concept used is transposing a matrix using 2D arrays and nested loops.

Program:

```
#include <stdio.h>
int main()
{
    printf("Name: Akshyat\n");
    printf("Roll No: 2510991394\n\n");
    int rows, cols;
    // Taking number of rows and columns from user
    printf("Enter the number of rows and columns of the matrix: ");
    scanf("%d %d", &rows, &cols);
    // Declaring matrix
    int arr[rows][cols];
    // Input matrix elements
    printf("Enter %d elements in the matrix:\n", rows * cols);
    for(int i = 0; i < rows; i++)
    {
        for(int j = 0; j < cols; j++)
        {
            scanf("%d", &arr[i][j]);
        }
    }
    // Displaying original matrix
```

```
    printf("\nOriginal Matrix:\n");

    for(int i = 0; i < rows; i++)

    {

        for(int j = 0; j < cols; j++)

        {

            printf("%d ", arr[i][j]);

        }

        printf("\n");

    }

    printf("\nTranspose Matrix:\n");

    for(int i = 0; i < cols; i++)

    {

        for(int j = 0; j < rows; j++)

        {

            printf("%d ", arr[j][i]);

        }

    }

    return 0;

}
```

Output Screenshot:



```
Name: Akshyat
Roll No: 2510991394

Enter the number of rows and columns of the matrix: 2 2
Enter 4 elements in the matrix:
1 4 7 11

Original Matrix:
1 4
7 11

Transpose Matrix:
1 7 4 11
PS D:\Official\Chitkara\1-Year\Sem-2\C - 1\functions>
```

Experiment No. 20.

Aim: Write a menu driven C program to show the use of in-built string functions like strlen, strcat, strcpy, strcmp, strrev etc.

Concept Used: The concept used is string handling in C using built-in functions like strlen, strcat, strcpy, strcmp, and strrev.

Program:

```
 #include <stdio.h>

#include <string.h>

int main(){

    char str1[100], str2[100];

    int choice;

    // Taking input strings

    printf("Enter first string: ");

    gets(str1);

    printf("Enter second string: ");

    gets(str2);

    // Menu Driven Program

    printf("\n----- STRING FUNCTIONS MENU -----\n");

    printf("1. Find Length of String\n");

    printf("2. Concatenate Strings\n");

    printf("3. Copy String\n");

    printf("4. Compare Strings\n");

    printf("5. Reverse String\n");

    printf("\nEnter your choice: ");

    scanf("%d", &choice);

    switch(choice)
```

```
{
  case 1:
    // strlen() returns length of string
    printf("\nLength of first string = %lu", strlen(str1));
    printf("\nLength of second string = %lu", strlen(str2));
    break;
  case 2:
    // strcat() joins two strings
    strcat(str1, str2);
    printf("\nConcatenated String = %s", str1);
    break;
  case 3:
    // strcpy() copies one string into another
    strcpy(str2, str1);
    printf("\nAfter Copy, Second String = %s", str2);
    break;
  case 4:
    // strcmp() compares two strings
    if(strcmp(str1, str2) == 0)
      printf("\nBoth strings are equal.");
    else
      printf("\nStrings are not equal.");
    break;
  case 5:
    // strrev() reverses the string
```

```c
        printf("\nReverse of first string = %s", strrev(str1));

        break;

      default:

        printf("\nInvalid Choice!");

  }

  return 0;

}
```

Output Screenshot:

```
Name: Akshyat
Roll No: 2510991394

Enter first string: Akshyat
Enter second string: Kumar

----- STRING FUNCTIONS MENU -----
1. Find Length of String
2. Concatenate Strings
3. Copy String
4. Compare Strings
5. Reverse String

Enter your choice: 1

Length of first string = 7
Length of second string = 5
PS D:\Official\Chitkara\1-Year\Sem-2\C - 1\functions>
```

Experiment No. 21.

Aim: Write a program to check that the given number is prime, Armstrong or perfect using the concept of functions.

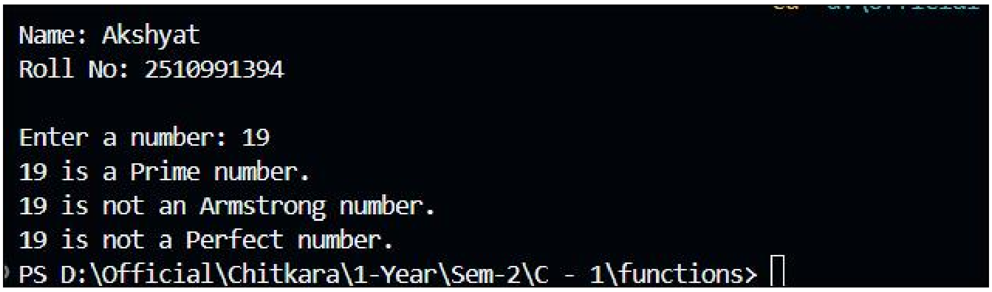
Concept Used: The concept used is user-defined functions to check prime, Armstrong, and perfect numbers.

Program:

```
#include <stdio.h>

// Function to check Prime Number

void prime(int num)

{

    int i, flag = 0;

    if(num <= 1)

    {

        printf("%d is not a Prime number.\n", num);

        return;

    }

    for(i = 2; i <= num / 2; i++)

    {

        if(num % i == 0)

        {

            flag = 1;

            break;

        }

    }

    if(flag == 0)

        printf("%d is a Prime number.\n", num);
```

```
    else
        printf("%d is not a Prime number.\n", num);
}
// Function to check Armstrong Number
void armstrong(int num)
{
    int temp, remainder, sum = 0;
    temp = num;
    while(temp != 0)
    {
        remainder = temp % 10;
        sum = sum + (remainder * remainder * remainder);
        temp = temp / 10;
    }
    if(sum == num)
        printf("%d is an Armstrong number.\n", num);
    else
        printf("%d is not an Armstrong number.\n", num);
}
void perfect(int num)
{
    int i, sum = 0;
    for(i = 1; i < num; i++)
    {
        if(num % i == 0)
```

```
        {
            sum = sum + i;
        }
    }
    if(sum == num)
        printf("%d is a Perfect number.\n", num);
    else
        printf("%d is not a Perfect number.\n", num);
}
int main()
{
    printf("Name: Akshyat\n");
    printf("Roll No: 2510991394\n\n");
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    prime(num);
    armstrong(num);
    perfect(num);
    return 0;
}
```
Output Screenshot:



```
Name: Akshyat
Roll No: 2510991394

Enter a number: 19
19 is a Prime number.
19 is not an Armstrong number.
19 is not a Perfect number.
PS D:\Official\Chitkara\1-Year\Sem-2\C - 1\functions>
```

Experiment No. 22.

Aim: Write a program to calculate the area and circumference of a circle using functions.

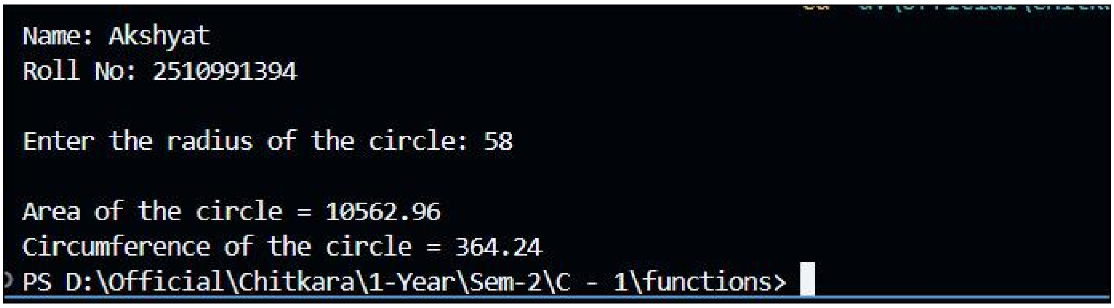
Concept Used: This program is based on the concept of user-defined functions in C. A function is a block of code that performs a specific task and can be reused whenever needed.

Program:

```
#include <stdio.h>

// Function to calculate area of circle

float area(float r)

{

   return 3.14 * r * r;

}

// Function to calculate circumference of circle

float circumference(float r)

{

   return 2 * 3.14 * r;

}

int main()

{

   printf("Name: Akshyat\n");

   printf("Roll No: 2510991394\n\n");

   float radius, a, c;

   // Taking radius as input

   printf("Enter the radius of the circle: ");

   scanf("%f", &radius);

   // Function calls
```

```c
    a = area(radius);

    c = circumference(radius);

    // Displaying results

    printf("\nArea of the circle = %.2f", a);

    printf("\nCircumference of the circle = %.2f", c);

    return 0;

}
```

Output Screenshot:



Experiment No. 23.

Aim: Write a program to swap two variables using the concept of call by value and call by reference.

Concept Used: This program is based on the concept of call by value and call by reference using functions in C. In call by value, a copy of variables is passed to the function, so original values do not change. In call by reference, the addresses of variables are passed using pointers, so the original values get changed after swapping. The program also uses functions, pointers, and swapping logic.

Program:

```
 #include <stdio.h>

// Function for Call by Value

void swapValue(int x, int y)

{

    int temp;

    temp = x;

    x = y;

    y = temp;

    printf("\nAfter swapping in Call by Value:");

    printf("\nx = %d  y = %d\n", x, y);

}

// Function for Call by Reference

void swapReference(int *x, int *y)

{

    int temp;

    temp = *x;

    *x = *y;

    *y = temp;
```

```
    printf("\nAfter swapping in Call by Reference:");
    printf("\nx = %d  y = %d\n", *x, *y);
}
int main()
{
    printf("Name: Akshyat\n");
    printf("Roll No: 2510991394\n\n");
    int a, b;
    // Input from user
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    // Original values
    printf("\nBefore swapping:");
    printf("\na = %d  b = %d\n", a, b);
    // Call by Value
    swapValue(a, b);
    // Values remain unchanged in main()
    printf("\nValues in main after Call by Value:");
    printf("\na = %d  b = %d\n", a, b);

    // Call by Reference
    swapReference(&a, &b);
    // Values changed in main()
    printf("\nValues in main after Call by Reference:");
    printf("\na = %d  b = %d\n", a, b);
```

```
    return 0;

}
```

Output Screenshot:

```
Name: Akshyat
Roll No: 2510991394

Enter two numbers: 56 34

Before swapping:
a = 56  b = 34

After swapping in Call by Value:
x = 34  y = 56

Values in main after Call by Value:
a = 56  b = 34

After swapping in Call by Reference:
x = 34  y = 56

Values in main after Call by Reference:
a = 34  b = 56
```

Experiment No. 24.

Aim: Write a program to show the use of passing pointer as arguments to the functions.

Concept Used: The concept used is passing pointers as function arguments to directly access and modify variable values.

Program:

```
#include <stdio.h>

// Function that receives pointers as arguments

void display(int *a, int *b)

{

    printf("\nValue of first number = %d", *a);

    printf("\nValue of second number = %d", *b);

    // Modifying values using pointers

    *a = *a + 10;

    *b = *b + 20;

}

int main()

{

    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");

    int x, y;

    // Input from user

    printf("Enter two numbers: ");

    scanf("%d %d", &x, &y);

    // Function call by passing addresses

    display(&x, &y);
```

```
    // Displaying modified values

    printf("\n\nAfter modification:");

    printf("\nx = %d", x);

    printf("\ny = %d", y);

    return 0;

}
```

Output Screenshot:

```
Name: Akshyat
Roll No: 2510991394

Enter two numbers: 56 34

Value of first number = 56
Value of second number = 34

After modification:
x = 66
y = 54
```

Experiment No. 25.

Aim: Write a program to find the factorial of a number by using the concept of recursion.

Concept Used: This program is based on the concept of recursion in C. In recursion, a function calls itself repeatedly until a stopping condition is reached. Here, the factorial() function calls itself to calculate the factorial of a number. It also uses functions, conditional statements, and multiplication operations.

Program:

```
#include <stdio.h>

// Recursive function to find factorial

int factorial(int n)

{

    if(n == 0 || n == 1)

        return 1;

    else

        return n * factorial(n - 1);

}

int main()

{

    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");

    int num;

    int fact;

    // Input from user

    printf("Enter a number: ");

    scanf("%d", &num);

    // Function call
```

```
    fact = factorial(num);

    // Display result

    printf("Factorial of %d = %d", num, fact);

    return 0;

}
```

Output Screenshot:

Experiment No. 26.

Aim: Write a Program in C to display the total number of appearances of a substring provided as input by the user in a given string.

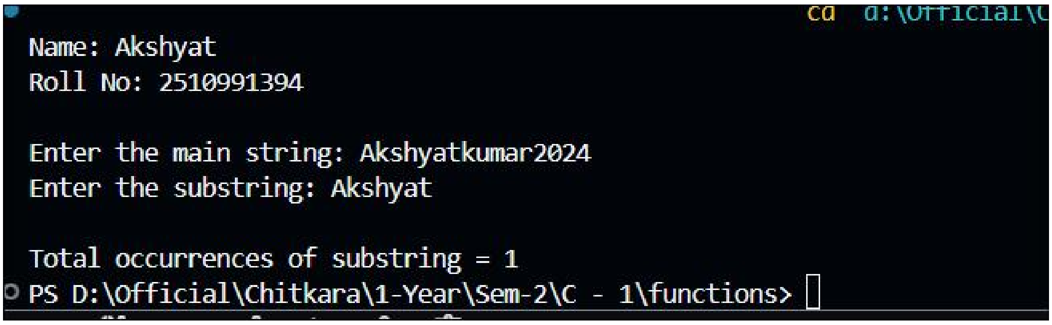
Concept Used: The concept used is string handling and substring searching to count appearances of a substring.

Program:

```
#include <stdio.h>

#include <string.h>

int main()

{

   printf("Name: Akshyat\n");

   printf("Roll No: 2510991394\n\n");

   char str[100], sub[100];

   int i, j, count = 0;

   // Input main string

   printf("Enter the main string: ");

   gets(str);

   // Input substring

   printf("Enter the substring: ");

   gets(sub);

   int strLen = strlen(str);

   int subLen = strlen(sub);

   // Checking substring occurrences

   for(i = 0; i <= strLen - subLen; i++)

   {

      for(j = 0; j < subLen; j++)
```

```
        {
            if(str[i + j] != sub[j])
                break;
        }
        // If complete substring matches
        if(j == subLen)
        {
            count++;
        }
    }
    // Display result
    printf("\nTotal occurrences of substring = %d", count);
    return 0;
}
```

Output Screenshot:



Experiment No. 27.

Aim: Write a program to display the sum of the digits of a number by using the concept of recursion.

Concept Used: The concept used is recursion to calculate the sum of digits of a number.

Program:

```
#include <stdio.h>

// Recursive function to find sum of digits

int sumDigits(int n)

{

    if(n == 0)

        return 0;

    else

        return (n % 10) + sumDigits(n / 10);

}

int main()

{

    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");

    int num, sum;

    // Input from user

    printf("Enter a number: ");

    scanf("%d", &num);

    // Function call

    sum = sumDigits(num);

    // Display result
```

```
    printf("Sum of digits = %d", sum);

    return 0;

}
```

Output Screenshot:

Experiment No. 28.

Aim: Write a C program to add two distances in inch & feet using the concept of structures.

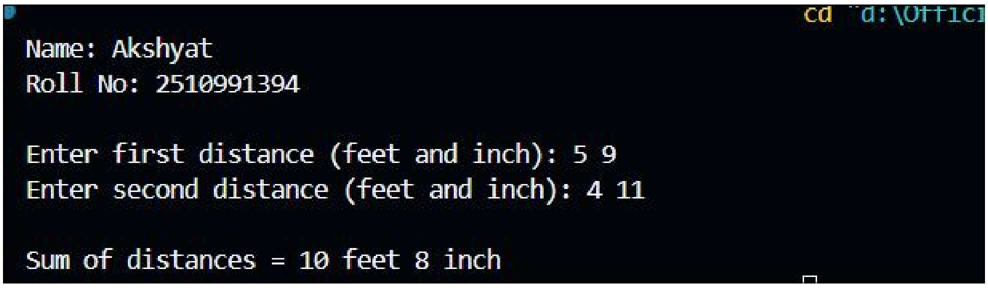
Concept Used: The concept used is structures in C to add distances in feet and inches.

Program:

```
#include <stdio.h>
// Structure declaration
struct Distance
{
    int feet;
    int inch;
};
int main()
{
    printf("Name: Akshyat\n");
    printf("Roll No: 2510991394\n\n");
    struct Distance d1, d2, sum;
    // Input first distance
    printf("Enter first distance (feet and inch): ");
    scanf("%d %d", &d1.feet, &d1.inch);
    // Input second distance
    printf("Enter second distance (feet and inch): ");
    scanf("%d %d", &d2.feet, &d2.inch);
    // Adding inches and feet
    sum.inch = d1.inch + d2.inch;
    sum.feet = d1.feet + d2.feet;
```

```
    // Converting inches into feet if inches >= 12
    if(sum.inch >= 12)
    {
        sum.feet = sum.feet + (sum.inch / 12);
        sum.inch = sum.inch % 12;
    }
    // Display result
    printf("\nSum of distances = %d feet %d inch",
           sum.feet, sum.inch);
    return 0;
}
```

Output Screenshot:



```
Name: Akshyat
Roll No: 2510991394

Enter first distance (feet and inch): 5 9
Enter second distance (feet and inch): 4 11

Sum of distances = 10 feet 8 inch
```

Experiment No. 29.

Aim: Write a C program to add two complex numbers using the concept of structures in C.

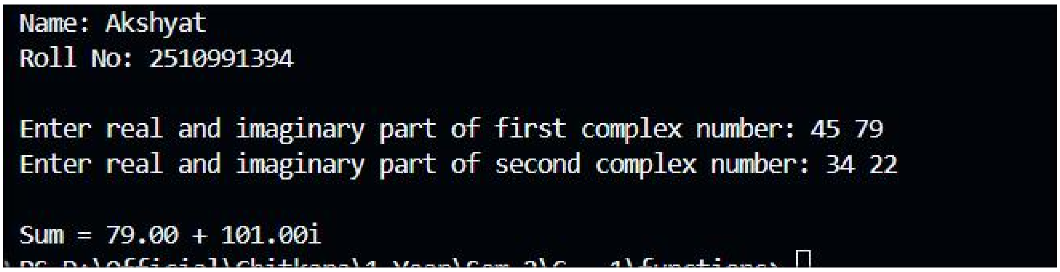
Concept Used: The concept used is structures in C to add complex numbers (real and imaginary parts).

Program:

```
#include <stdio.h>

// Structure declaration for complex number

struct Complex

{

    float real;

    float imaginary;

};

int main()

{

    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");

    struct Complex c1, c2, sum;

    // Input first complex number

    printf("Enter real and imaginary part of first complex number: ");

    scanf("%f %f", &c1.real, &c1.imaginary);

    // Input second complex number

    printf("Enter real and imaginary part of second complex number: ");

    scanf("%f %f", &c2.real, &c2.imaginary);

    // Adding complex numbers

    sum.real = c1.real + c2.real;
```

```
    sum.imaginary = c1.imaginary + c2.imaginary;

    // Display result

    printf("\nSum = %.2f + %.2fi",

           sum.real, sum.imaginary);

    return 0;

}
```

Output Screenshot:



Experiment No. 30.

Aim: Write a program in C to store the information of five employees using both concepts i.e. array of structure and array within structure.

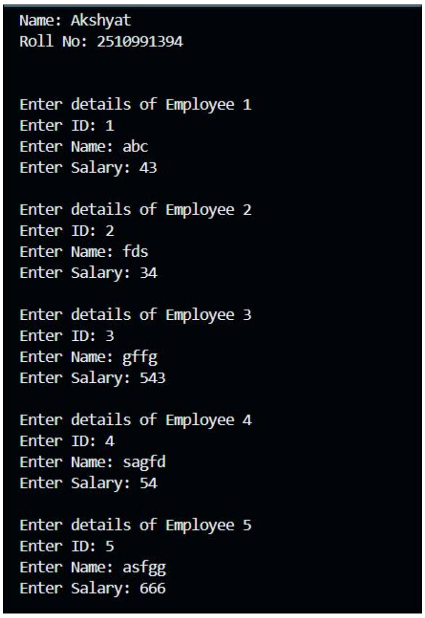
Concept Used: The concept used is arrays of structures and nested structures to store employee information.

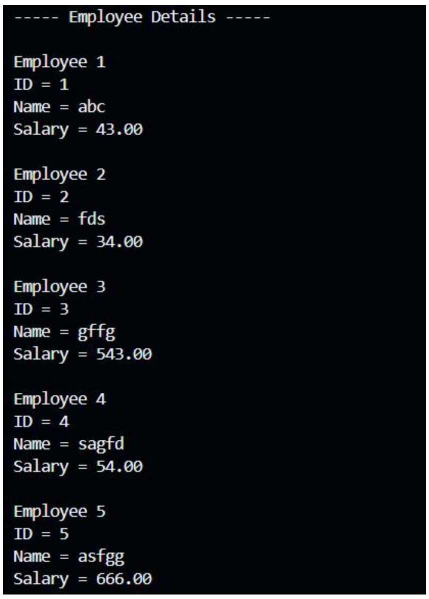
Program:

```
#include <stdio.h>

// Structure declaration

struct Employee

{

    int id;

    char name[50];     // Array within structure

    float salary;

};

int main()

{

    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");

    // Array of structures

    struct Employee emp[5];

    // Input employee details

    for(int i = 0; i < 5; i++)

    {

        printf("\nEnter details of Employee %d\n", i + 1);

        printf("Enter ID: ");

        scanf("%d", &emp[i].id);
```

```
        printf("Enter Name: ");

        scanf("%s", emp[i].name);

        printf("Enter Salary: ");

        scanf("%f", &emp[i].salary);

    }

    // Display employee details

    printf("\n----- Employee Details -----\n");

    for(int i = 0; i < 5; i++)

    {

        printf("\nEmployee %d\n", i + 1);

        printf("ID = %d\n", emp[i].id);

        printf("Name = %s\n", emp[i].name);

        printf("Salary = %.2f\n", emp[i].salary);

    }

    return 0;}
```

Output Screenshot:





Experiment No. 31.

Aim: Write a Program in C to find and replace a specific string in a file and also display the total number of appearances of that string.

Concept Used: The concept used is file handling in C to find and replace strings in a file and count their occurrences.

Program:

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

int main()

{

    printf("Name: Akshyat\n");

    printf("Roll No: 2510991394\n\n");

    FILE *fp1, *fp2;

    char fileName[100];

    char search[50], replace[50];

    char line[200];

    int count = 0;

    printf("Enter file name: ");

    scanf("%s", fileName);

    printf("Enter string to search: ");

    scanf("%s", search);

    printf("Enter string to replace: ");

    scanf("%s", replace);

    fp1 = fopen(fileName, "r");
```

```
if(fp1 == NULL)
{
    printf("Error opening file!\n");
    return 1;
}
fp2 = fopen("temp.txt", "w");
// Read file line by line
while(fgets(line, sizeof(line), fp1) != NULL)
{
    char *pos;
    // Search and replace in each line
    while((pos = strstr(line, search)) != NULL)
    {
        count++;
        // Print part before found string
        *pos = '\0';
        fprintf(fp2, "%s", line);
        // Print replacement string
        fprintf(fp2, "%s", replace);
        // Move pointer forward
        strcpy(line, pos + strlen(search));
    }
    fprintf(fp2, "%s", line);
}
fclose(fp1);
```

```
    fclose(fp2);

    // Replace original file with updated file

    remove(fileName);

    rename("temp.txt", fileName);

    printf("\nTotal occurrences of '%s' = %d\n", search, count);

    printf("File updated successfully!\n");

    return 0;

}
```

Output Screenshot:

```
Name: Akshyat
Roll No: 2510991394

Enter file name: abc.txt
Enter string to search: Akshyat
Enter string to replace: Kumar
Error opening file!
PS D:\Official\Chitkara\1-Year\Sem-2\C - 1\functions>
```